

Extraction routine

Overview

The Brute_extract method

Running main.py

The brute extract takes in a fits file.

- If you wish to rerun the full tracing routine then set the flat flag to true **However, this has not been set up to point to where the new traces are generated and will run the rest of the routine using the master traces.**
1. Grabs the master set of pkl trace files stores in llamas_pyjamas/mastercalib which have been generated using file 2024-11-26T23_41_55_mef.fits
 2. Sets up the package path and the runtimeenv if ray is used (currently only the tracing needs this)
 3. open the fits hdu
 4. if flat generate new traces, else grabs the mastercalib traces
 5. generates a zipped list of each hdu extension and its corresponding trace file name
 6. loops through each pair and grabs the header and the bias (nan median).
 1. opens the trace file
 2. create Extraction class obj using the trace and hdu data - bias
 3. append the extraction obj to a list
 7. load the list of extracted objects
 8. Define a new whitelight image name based on the original fitsfile
 9. Feed the loaded extractions list to the WhitelightFits function to generate a new WhiteLight.fits file in the output folder

ExtractLlamas

Inputs: trace_obj, hdu_data, hdu_header (as a dict for future ray implementation)

1. Grabs the colour, bench and side from the header and set them as class attributes
2. Create an empty array in the correct shape for the counts

3. Create an x range array based on naxis length and a 2D array using range naxis1 naxis2
4. start looping through each fibre in the trace object:
 1. if optimal is true:
 - a. x_spec, f_spec, weights = isolate_profile(fiber)
 - b. create empty array of length naxis1
 - c. for i in range naxis1: create a boolean mask for current x position, calculates weighted average for the non-masked pixels
 - d. Append to the fiber location in the counts array
 2. Else, use the boxcar method

By default the Extraction uses the optimal method unless set to false when the class is initiated

Isolate profile

Takes in a fibre number

1. creates a variable inprof which is equal to the fiberimg for the corresponding input fibre number
2. creates the profile variable which is the profimg from the trace that has been masked by the inprofile
3. sets x_spec, f_spec, weights as the 2D (naxis1 x naxis2) array masked by inprof, the data masked by inprof, and the profimg masked by inprof

LUT

I have generated a LUT table using the 2024-11-26T23_41_55_mef.fits

- The original set of these is in traceLUT_orig.json
- If you wish to make a new one:
 1. move the traceLUT.json file to one named backup and create a copy of traceLUT_template.json as traceLUT.json so that you are working with a fresh file

I have a script which can flip the fib_pos order based on the colour and benchside called flip_positions under utils.py